

Arquitectura del Sistema de Control con Lógica Difusa

Equipo de Desarrollo

28 de agosto de 2025

1. Introducción

Este documento detalla la arquitectura y el flujo de control del sistema de lógica difusa, un motor que combina la flexibilidad del razonamiento fuzzy con mecanismos de seguridad y una ejecución determinista. El sistema está diseñado para operar en ciclos de un minuto, coordinando tres componentes principales: el **fuzzy-service**, el **actuator-service**, y el **firmware** en el ESP32.

El objetivo principal es asegurar un control flexible, seguro y determinista, donde el **fuzzy-service** actúa como el cerebro interpretativo, el **actuator-service** garantiza la correcta ejecución y secuencialidad de los comandos, y las **reglas de control directas** funcionan como un cinturón de seguridad.

2. Flujo de Operación General

El sistema de control basado en lógica difusa (Mamdani puro) funciona en ciclos de un minuto. En cada ciclo, se recogen y validan los datos de los sensores. Con esos valores, el *fuzzy-service* evalúa el conjunto de reglas difusas. Cada regla tiene como antecedente una combinación de condiciones sobre variables de entrada y como consecuente una rutina, es decir, un conjunto de acciones sobre uno o varios actuadores.

Estas acciones ya llegan resueltas con parámetros concretos: pin físico, potencia (ON/OFF o PWM) y un tiempo de expiración absoluto (*timestamp_expiration*) en lugar de una duración relativa. Un comando se interpreta como "mantener ON hasta $T = \text{now} + X$ ", no como "mantener ON durante X ".

2.1. Mecanismos de Control y Prioridad

- **Motor Difuso:** El *fuzzy-service* registra en su log qué reglas se dispararon, qué valores de sensores se usaron y qué acciones se derivaron. Luego envía al *actuator-service* un *payload* con dichas acciones.
- **Reglas de Seguridad:** El sistema incorpora reglas de control directas que tienen prioridad absoluta sobre cualquier rutina fuzzy. Ejemplos:
 - Si *TemperaturaAgua* > 30°C, apagar el CalefactorAgua.
 - Si *HumedadRelativa* > 90 %, apagar el Humidificador.
 - Si *NivelAgua* < *MínimoSeguro*, apagar la Motobomba.
- **Manejo de Actuadores:** El *actuator-service* mantiene una cola de ejecución por actuador. Si un comando nuevo llega antes de que el actuador ejecute el previo, lo reemplaza. Si ya está en ejecución, se ajusta la potencia o expiración sin acumular tiempos residuales.

2.2. Coordinación de Rutinas

- **Extensión de Rutinas:** Si una rutina activa tiene más de dos pasos y el primero sigue activo, puede extenderse si es del mismo tipo. De lo contrario, la nueva rutina se encola.
- **Gestión de Recursos:** Si una rutina requiere un actuador ya en uso, debe esperar en la cola. Si no hay conflictos, las rutinas pueden iniciarse en paralelo.
- **Condiciones de Activación:** Si un sensor está desactivado, las reglas relacionadas se suspenden. La activación de iluminación artificial requiere de varias lecturas consecutivas y se apaga forzosamente a las 20:00 h.

3. Ciclo de Vida de una Rutina

3.1. Evaluación Fuzzy

Cada minuto, el `fuzzy-service` evalúa los sensores y, al dispararse una regla, genera una rutina independiente. Se realiza una solicitud POST al `actuator-service` con la rutina completa, incluyendo pasos y expiraciones absolutas.

3.2. Registro de un sistema fuzzy

Esta colección Agrupa un conjunto de variables y reglas que permiten conocer las diferentes entradas, reglas, y salidas con las que cuenta un sistema fuzzy.

Listing 1: Registro de una sistema fuzzy

```
{
  "_id": "fuzzy_1", // mongo id
  "name": "Vegetativo Día", // nombre publico para front
  "status": "in_use", // in_use | reserved
  "operators": { "and": "min", "or": "max", "not": "complement" },
  "defuzzMethod": "centroid",
  "variables": [lsdkfjladskfv23,oirlskdfl2314] // mongo id de la
    variable
  "rules": [aofiualekfj,aofiualekfj34342] // mongo id de la regla
  "createdBy": "fuzzy-service", // no se si dejarlo
  "createdAt": "20250812T100000Z",
  "updatedAt": "20250812T150000Z"
}
```

3.3. Registro de Variables en fuzzy

Esta colección especifica cada una de las variables de entrada y salida registrados en ese grupo fuzzy y que permiten conocer las etiquetas asociadas a esta.

Listing 2: Registro de una variable

```
{
  "_id": "var_temp_air", // id mongo
  "systemId": "fuzzy_1", // id mongo
  "name": "temp_air",
  "type": "input",
}
```

```
"sensor_id": "5783qhgg445tu" // id mongo de la colección de
    sensores
"termIds": ["term_temp_low", "term_temp_ok", "term_temp_high"]
}
```

3.4. Registro de etiquetas de una variable fuzzy

Esta colección especifica cada una de las etiquetas que tiene una variable, en esta se da a conocer la función de membresía seleccionada para esta etiqueta y el rango de cada uno de sus lados.

Listing 3: Registro de una etiqueta

```
{
  "_id": "term_temp_high", //id mongo
  "label": "high", //nombre de etiqueta
  "mf": {
    "type": "trapezoid", //tipo de función
    "params": [28, 30, 40, 40] // a,b,c,d
  }
}
```

3.5. Registro en actuator-service

El `actuator-service` consulta su base de datos para obtener los detalles de cada actuador (como el pin físico y el modo de operación) y valida la coherencia entre el comando recibido y el actuador. Luego crea un registro de la rutina con su estado inicial en *scheduled* y reenvía la rutina al ESP32.

3.6. Ejecución en el Firmware (ESP32)

El firmware gestiona las rutinas activas, resolviendo conflictos y aplicando extensiones. Si una rutina es extendida o reemplazada, el ESP32 actualiza sus temporizadores y parámetros.

3.7. Finalización de la Rutina

Una vez que todos los pasos de la rutina han concluido, el ESP32 la marca como **finalizada** y envía un mensaje de confirmación al `actuator-service`.

Listing 4: Mensaje de confirmación del ESP32

```
{
  "routineId": "recirculaAgua_20250825T1500",
  "status": "completed",
  "finishedAt": "2025-08-25T15:01:30Z",
  "results": [
    { "pin": 12, "status": "ok" },
    { "pin": 27, "status": "cancelled" }
  ],
  "logs": [
    "Se extendió el tiempo del paso 2 a 15:02 por rutina X",
    "Se cancelo paso 3 porque rutina Y lo sobrescribio"
  ]
}
```

3.8. Actualización del Log

El `actuador-service` actualiza el registro de la rutina con el estado final (calculado a partir de los resultados de cada paso), la hora de finalización y los logs de ejecución, cerrando el ciclo.

4. Estructuras de Datos y Contratos entre Servicios

4.1. Estructura de Actuadores en la Base de Datos

Este es el esquema que el `actuador-service` utiliza para registrar los actuadores, incluyendo el pin físico y el modo de operación, información clave para la validación y el envío de comandos al firmware.

Listing 5: Estructura de un actuador en la base de datos

```
{
  "_id": { "$oid": "68ab8bb77b05e378b3731341" },
  "Esp32Id": "6883fff7b079309f3ba4f238",
  "Name": "Ventiladores 12V",
  "Type": "Fan",
  "PhysicalId": "FANS-001",
  "Pin": 16,
  "Mode": "PWM", // DIGITAL | PWM
  "Status": "Active",
  "Location": "rack-1",
  "CreatedAt": { "$date": { "$numberLong": "1756072887865" } }
}
```

4.2. Estructura de Reglas (fuzzy-service)

Define cómo se almacenan las reglas difusas.

Listing 6: Estructura de una regla fuzzy

```
{
  "ruleId": "aofiualekfj",
  "name": "low_ec"
  "systemId": "fuzzy_1",
  "description": "Si EC es baja, activar recirculación de agua",
  "conditions": [
    {
      "sensor": "EC",
      "operator": "IS",
      "value": "Baja"
    },
    { "sensor": "NivelAgua", "operator": "NOT", "value": "Bajo" }
  ],
  "conector": ["AND"]
  "consequent": "RecirculaAgua", //aquí va el routine id
  "createdAt": "2025-08-25T15:00:00Z"
}
```

4.3. Estructura de una rutina (fuzzy-service)

Define cómo se almacenan las rutinas asociando cada actuador con una etiqueta que permite calcular por defuzificación la potencia y la duración de un paso según las funciones de membresía asociadas a esta.

Listing 7: Estructura de una rutina fuzzy

```

{
  "routineId": "aofuhakjfbv123124325wsf",
  "routine_name": "Calentar Invernadero",
  "steps": [
    {
      "actuator": "68ab8bb77b05e378b3731341",
      "power_tag_id": "askfjalkrjesf34", // id de tag -> ej: "medio"
      "duration_tag_id": "askfjalkrjesf34" // id de tag -> ej: "
        medio"
    },
    {
      "actuator": "68ab8bb77b3123378b37311212",
      "power_tag_id": "askfjalkrje23w4", // id de tag -> ej: "Alto"
      "duration_tag_id": "askfjal2aszrsf34" // id de tag -> ej: "
        Alto"
    }
  ]
}

```

4.4. Payload recibido por actuador-service

Formato del mensaje enviado por el `fuzzy-service` al `actuador-service`, donde las acciones se definen por el ID del actuador y la potencia o ciclo de trabajo. El `fuzzy-service` no necesita conocer los detalles de hardware.

Listing 8: Payload recibido en el actuador-service

```

[
  {
    "routineId": "recirculaAgua",
    "steps": [
      {
        "actuator": "68ab8bb77b05e378b3731341",
        "power": "ON",
        "expiration": "2025-08-25T15:01:00Z"
      },
      {
        "actuator": "68ab8bb77b3123378b37311212",
        "dutyCycle": 70,
        "expiration": "2025-08-25T15:02:00Z"
      }
    ]
  },
  {
    "routineId": "Calentar Invernadero",
    "steps": [

```

```

    {
      "actuator": "68ab8bb77b05e378b3731341",
      "power": "ON",
      "expiration": "2025-08-25T15:01:00Z"
    },
    {
      "actuator": "68ab8bb77b3123378b37311212",
      "dutyCycle": 70,
      "expiration": "2025-08-25T15:02:00Z"
    }
  ]
}
]

```

4.5. Esquema de registro en fuzzy-service sobre entradas y la regla activada

Este es el registro interno del proceso realizado en **fuzzy-service** cuando se recibe una lectura cada minuto. Se almacenan las lecturas recibidas y las reglas activadas.

Listing 9: Esquema de registro en fuzzy-service

```

{
  "evalId": "uuid-12345",
  "systemId": "fuzzy-v1",
  "timestamp": "2025-08-25T15:00:00Z",
  "inputs": {
    "ph": 6.5,
    "ec": 1.2,
    "air_temp": 28.3,
    "humidity": 55.2,
    "lux": 11000,
    "water_level": 12.5,
    "water_temp": 22.0
  },
  "activated_rules": [
    {
      "Rule ID": "calentarAmbiente",
      "firingStrength": 0.8,
      "output_values": [
        { "actuator_id": "adfjaldskf", "Power": 70, "Duration": 80 },
        { "actuator_id": "23480uhrkj236", "Power": ON, "Duration": 40 }
      ]
    },
    {
      "Rule ID": "RecircularAguar",
      "firingStrength": 0.8,
      "output_values": [
        { "actuator_id": "adfjaldskf", "Power": 70, "Duration": 80 },
        { "actuator_id": "23480uhrkj236", "Power": 50, "Duration": 30 }
      ]
    }
  ]
}

```

```

    ]
  }
]
}

```

4.6. Esquema de registro en actuator-service

Este es el registro interno del comando en el **actuator-service**. Se almacena una versión simplificada, sin duplicar los pasos de la rutina, y se actualiza con el estado general y los resultados de la ejecución.

Listing 10: Esquema de registro en actuator-service

```

{
  "commandId": "recirculaAgua_20250825T1500",
  "routineId": "recirculaAgua",
  "statusGeneral": "scheduled", // scheduled | running | completed |
    completed_with_warnings | failed | cancelled
  "createdAt": "2025-08-25T15:00:00Z",
  "finishedAt": "2025-08-25T15:01:30Z",
  "results": [
    { "pin": 12, "status": "ok" },
    { "pin": 27, "status": "cancelled", "executionLog": "Rutina de
      control 'apagar-bomba' interrumpió a los 12s" }
  ]
}

```

4.7. Payload para actuator-service a firmware (ESP32)

Formato ligero para la comunicación con el microcontrolador. El **actuator-service** enriquece el payload original con los pines físicos y el modo de operación ('DIGITAL' o 'PWM'). La expiración se convierte a **timestamp epoch (ms)**, un formato más eficiente para la gestión en el firmware.

Listing 11: Payload al firmware ESP32

```

{
  "esp32Id": "6883fff7b079309f3ba4f238",
  "jobSchedule": [
    {
      "queue": [
        {
          "commandId": "recirculacion-dd/mm/aaaa hh.mm.ss",
          "steps": [
            { "pin": 16, "power": 80, "duration": 30 },
            { "pin": 18, "power": 50, "duration": 10 }
          ]
        },
        {
          "commandId": "iluminacion-dd/mm/aaaa hh.mm.ss",
          "steps": [
            { "pin": 21, "power": 100, "duration": 120 }
          ]
        }
      ]
    }
  ]
}

```



```
]
},
{
  "queue": [
    {
      "commandId": "ventilacion-dd/mm/aaaa hh.mm.ss",
      "steps": [
        { "pin": 23, "power": 70, "duration": 45 }
      ]
    }
  ]
}
]
```

4.8. Confirmación del Firmware a actuator-service

Mensaje de respuesta del ESP32 al finalizar la rutina. Incluye los resultados por paso ('ok' o 'cancelled') y un array de logs para mayor trazabilidad, información crucial que el **actuator-service** usará para actualizar su registro.

Listing 12: Confirmación del firmware

```
{
  "esp32Id": "6883fff7b079309f3ba4f238",
  "commandId": "recirc-20250828T150000",
  "steps": [
    { "pin": 16, "status": "ok" },
    {
      "pin": 25,
      "status": "cancelled",
      "executionLog": "Rutina de control 'apagar-bomba' interrumpió a los 12s"
    },
    { "pin": 27, "status": "ok" }
  ]
}
```

5. Conclusiones

La arquitectura propuesta separa claramente las responsabilidades: el **fuzzy-service** define la lógica de alto nivel, el **actuator-service** orquesta y registra las rutinas, y el **firmware** en el ESP32 se encarga de la ejecución atómica y la gestión en tiempo real. Este diseño modular garantiza flexibilidad, seguridad y trazabilidad completa de cada acción del sistema, ****mejorando la solidez y el registro de la ejecución a través de la comunicación enriquecida entre servicios****.